Bayesian Modeling of Epidemic Intensity in Spatially Structured Bat Roosts

Henry Traynor

October 22, 2026

Abstract

Human-driven landscape modification can influence wildlife disease dynamics by changing host population size and spatial organization. Modern buildings may support larger populations than traditional structures, potentially altering epidemic intensity. I simulated epidemics on landscapes with a gradient of modern building presence and spatial clustering to determine the effect of these factors on epidemic intensity using a spatially explicit and density dependent movement process. Epidemic intensity was measured using peak prevalence, peak infected abundance, and time to peak. I used a Bayesian regression to estimate interactions between modernization, spatial structure, and peak prevalence.

Introduction

Landscape modification can alter wildlife disease dynamics by changing both host population density and the spatial distribution of individuals. In bats, modern buildings may provide roosts with greater occupancy capacity than traditional structures, potentially increasing the number of hosts available for pathogen transmission. However, the influence of roost modernization may depend on how roosts are distributed across the landscape, as spatial clustering can affect patterns of host movement and contact.

Understanding the relative contributions of host population size and spatial structure is important for predicting how human-driven landscape change influences epidemic dynamics. In this study, I used simulated epidemics to investigate how the proportion of modern building structures and the spatial aggregation of roosts influence epidemic intensity. I quantified epidemic intensity using peak infection prevalence and used Bayesian regression to estimate the interactions between landscape characteristics and epidemic behavior.

Methods

Spatial Model

I used a spatially explicit simulation model to investigate how the proportion of modern buildings and the spatial distribution of those roosts influence epidemic dynamics. The model represents bat populations distributed among discrete roosts across a two-dimensional landscape.

I varied the proportion of modern roosts (`prop_modern`) from 0 to 1 in increments of 0.25. Spatial structure was varied by specifying the number of clusters of roosts on the landscape and the expected standard deviation of the distance from a given roost to the center of the roost's assigned cluster. This gave me landscapes that were highly clustered, thus highly inter-connected, and landscapes that were distributed broadly across space.

To quantify the spatial arrangement of roosts, I calculated the Clark-Evans aggregation index (R) for each landscape. This statistic describes the spatial distribution of points relative to the expected distribution given complete spatial randomness. Lower values ($R < 1$) indicate greater clustering, higher values ($R > 1$) indicate greater dispersion, and $R \approx 1$ indicates complete spatial randomness.

Population Movement and Epidemic Dynamics

Individual bats were classified as susceptible, infected, or recovered (SIR). During the daytime phase of the model, infection dynamics were governed by standard SIR dynamics within each roost.

$$\frac{dS}{dt} = -\beta \frac{SI}{N}$$
$$\frac{dI}{dt} = \beta \frac{SI}{N} - \gamma I$$
$$\frac{dR}{dt} = \gamma I$$

where,

- N is the total number of bats a roost
- β is the rate at which disease is transmitted per contact between a susceptible and infectious bat
- γ is the rate at which infected bats recover

At night, bats dispersed across the landscape and returned to a roost, carrying their disease-state with them. Particular methods for bat movement can be found here on the Github repository for `RoostEpiPkg`, a package I developed to streamline the workflow of my thesis. This package can be downloaded using: `remotes::install_github("HenryTraynor/RoostEpiPkg")`. Vignettes are available for most of the functions in the package.

Each simulation was ran for 60 days. The transmission and recovery parameters were specified as $\beta = 0.5$ and $\gamma = 0.03$. The simulation recorded the number of susceptible, infected, and recovered bats in each roost over time, allowing epidemic outcomes to be summarized. I quantified epidemic intensity using peak prevalence P_{peak}, the maximum daily proportion of the total bat population that was infected.

$$P_{\text{peak}} = \max_t \left(\frac{\sum_j I_{jt}}{\sum_j N_{jt}} \right)$$

where I_{jt} is the number of infected bats in roost j on day t and N_{jt} is the total population in that roost on that day.

Bayesian Statistical Analysis

I used a Bayesian model in `JAGS` using the `jagsUI` package to evaluate the effects of landscape structure and building modernization on peak prevalence.

Since prevalence is a continuous proportion on $[0, 1]$, I modeled peak prevalence as a Beta-distributed response:

$$Y_i \sim \text{Beta}(\mu_i \phi_i, (1 - \mu_i)\phi)$$
$$\text{logit}(\mu_i) = \beta_0 + \beta_1 M_i + \beta_2 R_i + \beta_3 M_i R_i$$

where:

- Y_i is the peak prevalence
- M_i is `prop_modern`
- R_i is the standardized Clark-Evans spatial index
- ϕ is the precision parameter; higher values mean less residual variation

Data Generation

Defining Parameter Space

```
# Libraries
# Install first time with commented line below or for updates
#remotes::install_github("HenryTraynor/RoostEpiPkg")
library(RoostEpiPkg)

library(dplyr)
library(ggplot2)
library(progress)

# Paired spatial settings
spatial_settings <- data.frame(
  spatial_setting = 1:9,
  num_clusters = c(1, 1, 2, 3, 5, 8, 12, 20, 25),
  sd = c(0.02, 0.05, 0.04, 0.06, 0.08,
         0.10, 0.12, 0.15, 0.20)
)

# Modernization Gradient
prop_modern_values <- seq(0, 1, by = 0.25)

# Infection Parameters
beta <- 0.5
gamma <- 0.03

# Experiment Parameters
n_replicates <- 1
n_days <- 40
num_roosts <- 25
nx <- 50
ny <- 50

# All combinations of experimental conditions
parameter_grid <- expand.grid(
  prop_modern = prop_modern_values,
  spatial_setting = spatial_settings$spatial_setting,
  replicate = seq_len(n_replicates)
)

# TEMPORARY
parameter_grid_small <- parameter_grid[c(TRUE,FALSE),]

nrow(parameter_grid_small) # number of simulations
```

```
## [1] 23
```

```r
# Function for calculating metrix of CSR
calculate_clark_evans <- function(x, y) {
  coords <- cbind(x, y)
  n <- nrow(coords)

  if (n < 2) {
    return(NA_real_)
  }

  distances <- as.matrix(dist(coords))
  diag(distances) <- Inf

  observed_mean <- mean(apply(distances, 1, min))

  # Approximate expected mean nearest-neighbor distance
  # for complete spatial randomness in a unit-area domain.
  expected_mean <- 0.5 / sqrt(n)

  observed_mean / expected_mean
}
```

Simulating Across Parameter Space

```r
# Function to run one simulation for cleanliness
run_one_simulation <- function(
    prop_modern,
    spatial_setting,
    replicate,
    spatial_settings,
    num_roosts,
    nx,
    ny,
    n_days,
    beta,
    gamma
) {

  # Select spatial parameters
  setting <- spatial_settings[
    spatial_settings$spatial_setting == spatial_setting,
  ]

  nc <- setting$num_clusters
  cluster_sd <- setting$sd

  # Generate landscape and roost capacities
  roost_info <- create_roosts(
    num_roosts = num_roosts,
    num_clusters = nc,
    sd = cluster_sd,
```

```
  prop_modern = prop_modern
)

# Measure realized spatial structure
clark_evans_R <- calculate_clark_evans(
  roost_info$x,
  roost_info$y
)

# Initialize roost populations before running the simulation
roost_info <- initialize_roost_populations(
  roost_info
)

total_population_initial <- sum(roost_info$N)
total_capacity <- sum(roost_info$N_max)

# Set up movement and epidemic model
spatial_params <- create_spatial_params(
  nx = nx,
  ny = ny
)

movement_params <- setup_movement(
  roost_info,
  spatial_params
)

epidemic_params <- create_epidemic_params(
  beta = beta,
  gamma = gamma,
  model = "SIR"
)

# Run epidemic simulation
sim <- run_simulation(
  roost_info = roost_info,
  spatial_params = spatial_params,
  movement_params = movement_params,
  epidemic_params = epidemic_params,
  n_days = n_days
)

# Extract daily epidemic outcomes
daily_results <- bind_rows(
  lapply(seq_along(sim), function(day_index) {

    roosts <- sim[[day_index]]$roosts

    total_N <- sum(roosts$N)
    total_I <- sum(roosts$I)

    data.frame(
```

```
        day = day_index - 1L,
        total_population = total_N,
        total_infected = total_I,
        prevalence = if (total_N > 0) {
          total_I / total_N
        } else {
          NA_real_
        }
      )
    })
  )

  # Identify peak among days with valid prevalence
  valid_days <- which(is.finite(daily_results$prevalence))

  if (length(valid_days) == 0) {
    peak_prevalence <- NA_real_
    peak_infected <- NA_real_
    time_to_peak <- NA_integer_
  } else {
    peak_value <- max(
      daily_results$prevalence[valid_days]
    )

    peak_index <- valid_days[
      which(daily_results$prevalence[valid_days] == peak_value)[1]
    ]

    peak_prevalence <- daily_results$prevalence[peak_index]
    peak_infected <- daily_results$total_infected[peak_index]
    time_to_peak <- daily_results$day[peak_index]
  }

  # Record epidemic and landscape summary
  data.frame(
    prop_modern = prop_modern,
    spatial_setting = spatial_setting,
    num_clusters = nc,
    sd = cluster_sd,
    Clark_Evans_R = clark_evans_R,
    replicate = replicate,
    total_population_initial = total_population_initial,
    total_capacity = total_capacity,
    peak_infected = peak_infected,
    peak_prevalence = peak_prevalence,
    time_to_peak = time_to_peak
  )
}

#-------------------------
# SIMULATION
#-------------------------
```

```r
#install.packages(c("future", "future.apply", "parallelly", "progressr"))
library(future)
library(future.apply)
library(parallelly)
library(progressr)

# Set up parallel processing (leave one core free)
n_workers <- max(1, parallelly::availableCores() - 1)

plan(multisession, workers = n_workers)

handlers("progress")

with_progress({

  p <- progressor(steps = nrow(parameter_grid_small))

  results <- future_lapply(
    seq_len(nrow(parameter_grid_small)),
    function(i) {

      params <- parameter_grid_small[i, ]

      result <- tryCatch(
        run_one_simulation(
          prop_modern = params$prop_modern,
          spatial_setting = params$spatial_setting,
          replicate = params$replicate,
          spatial_settings = spatial_settings,
          num_roosts = num_roosts,
          nx = nx,
          ny = ny,
          n_days = 60,
          beta = beta,
          gamma = gamma
        ),
        error = function(e) {
          message(
            "Simulation ", i, " failed: ",
            conditionMessage(e)
          )
          NULL
        }
      )

      p()  # Update progress after this simulation finishes

      result
    },
    future.seed = TRUE
  )

})
```

```r
simulation_results <- bind_rows(results)

plan(sequential)

# Save data

# Create a folder for simulation output
dir.create("data")
# Save the results as RDS and CSV
saveRDS(
  simulation_results,
  file = "data/simulation_results_60days_small.rds"
)
write.csv(
  simulation_results,
  file = "data/simulation_results_60days_small.csv",
  row.names = FALSE
)
```

```r
simulation_results <- readRDS(
  "data/simulation_results_60days_small.rds"
)
```

Prepare Data for JAGS

```r
library(jagsUI)

dat <- simulation_results

# Standardize the spatial agg. index
dat$R_scaled <- as.numeric(scale(dat$Clark_Evans_R))

# center modernization around 0.5
dat$modern_c <- dat$prop_modern - 0.5

jags_data <- list(
  N = nrow(dat),
  y = dat$peak_prevalence,
  modern = dat$modern_c,
  spatial = dat$R_scaled
)
```

Fit model in JAGS

```r
model_string <- "
model {

  # Priors
  beta0 ~ dnorm(0, 0.01)
```

```
  beta_modern ~ dnorm(0, 0.01)
  beta_spatial ~ dnorm(0, 0.01)
  beta_interaction ~ dnorm(0, 0.01)

  # Beta precision
  phi ~ dgamma(2, 0.1)

  # Likelihood
  for(i in 1:N) {

    y[i] ~ dbeta(
      mu[i] * phi,
      (1- mu[i]) * phi
    )

    logit(mu[i]) <-
      beta0 +
      beta_modern * modern[i] +
      beta_spatial * spatial[i] +
      beta_interaction * modern[i] * spatial[i]
  }
}
"

writeLines(model_string, "beta_regression.jags")
```

```
# Initial values for each chain
inits <- function() {
  list(
    beta0 = 0,
    beta_modern = 0,
    beta_spatial = 0,
    beta_interaction = 0,
    phi = 10
  )
}

parameters <- c(
  "beta0",
  "beta_modern",
  "beta_spatial",
  "beta_interaction",
  "phi"
)

fit <- jags(
  data = jags_data,
  inits = inits,
  parameters.to.save = parameters,
  model.file = "beta_regression.jags",
  n.chains = 3,
  n.adapt = 1000,
  n.iter = 6000,
```

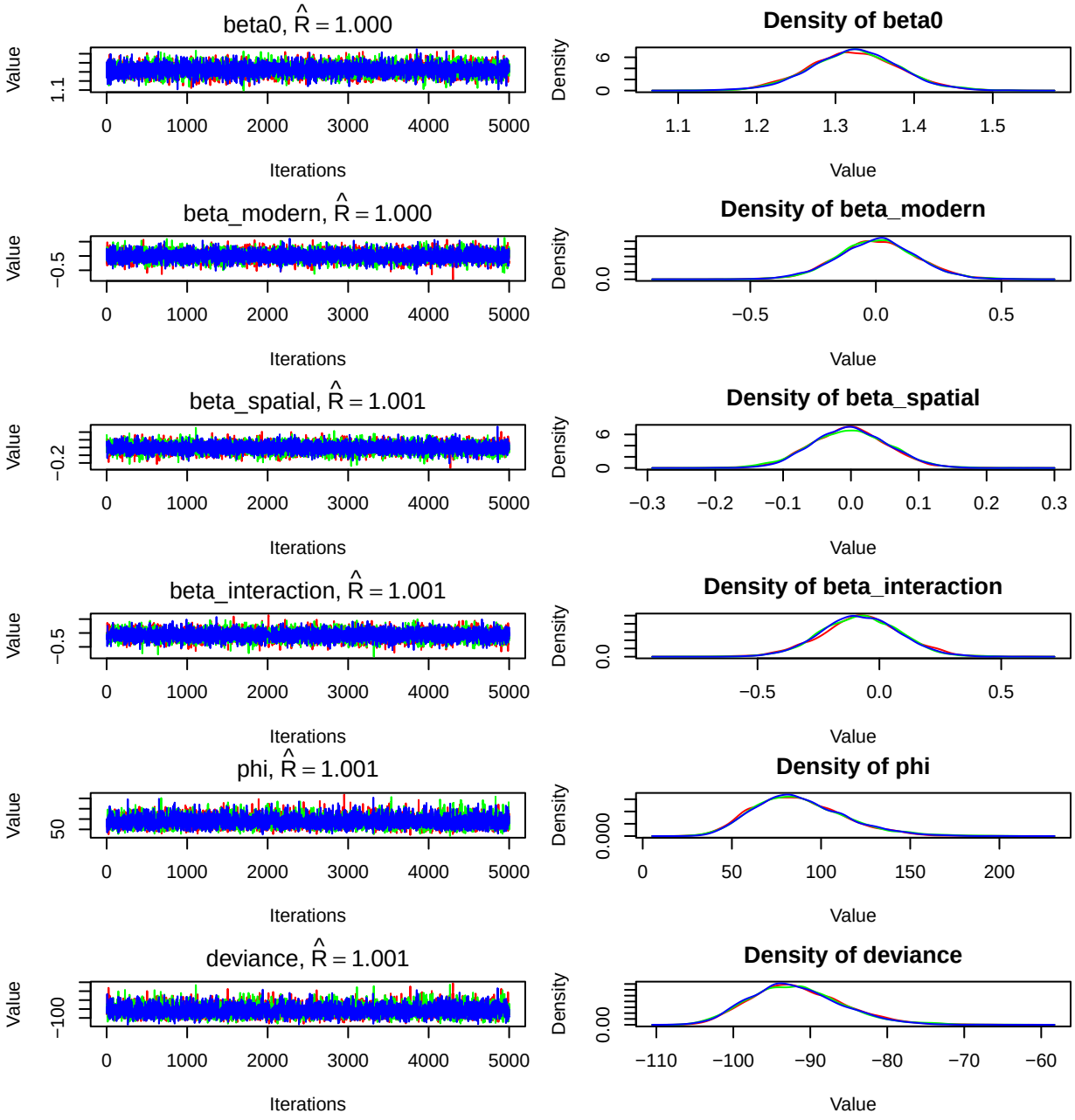
```
  n.burnin = 1000,
  parallel = TRUE
)
```

Results

```
print(fit)
```

```
## JAGS output for model 'beta_regression.jags', generated by jagsUI.
## Estimates based on 3 chains of 6000 iterations,
## adaptation = 1000 iterations (sufficient),
## burn-in = 1000 iterations and thin rate = 1,
## yielding 15000 total samples from the joint posterior.
## MCMC ran in parallel for 0.044 minutes at time 2026-10-22 15:57:05.438289.
##
##                      mean     sd     2.5%     50%   97.5% overlap0     f  Rhat
## beta0               1.325  0.057    1.212   1.325   1.438    FALSE 1.000 1.000
## beta_modern         0.006  0.158   -0.302   0.006   0.322     TRUE 0.514 1.000
## beta_spatial       -0.002  0.057   -0.113  -0.002   0.108     TRUE 0.517 1.001
## beta_interaction   -0.076  0.163   -0.400  -0.076   0.242     TRUE 0.684 1.001
## phi                88.032 25.521   45.944  85.301 145.258    FALSE 1.000 1.001
## deviance          -91.741  5.967 -101.893 -92.229 -78.775    FALSE 1.000 1.001
##                   n.eff
## beta0             10440
## beta_modern       15000
## beta_spatial       4734
## beta_interaction   5332
## phi                3650
## deviance           1788
##
## Successful convergence based on Rhat values (all < 1.1).
## Rhat is the potential scale reduction factor (at convergence, Rhat=1).
## For each parameter, n.eff is a crude measure of effective sample size.
##
## overlap0 checks if 0 falls in the parameter's 95% credible interval.
## f is the proportion of the posterior with the same sign as the mean;
## i.e., our confidence that the parameter is positive or negative.
##
## DIC info: (pD = var(deviance)/2)
## pD = 17.8 and DIC = -73.959
## DIC is an estimate of expected predictive error (lower is better).
```

```r
plot(fit)
```




```r
posterior_summary <- as.data.frame(fit$summary)

posterior_summary$parameter <- rownames(posterior_summary)

posterior_summary <- posterior_summary[
  ,
  c(
    "parameter",
    "mean",
    "sd",
```

```
    "2.5%",
    "50%",
    "97.5%",
    "Rhat",
    "n.eff"
  )
]

rownames(posterior_summary) <- NULL

print(posterior_summary)
```

```
##          parameter          mean          sd         2.5%          50%
## 1            beta0   1.324512515  0.05703288    1.2121808   1.324549504
## 2      beta_modern   0.006005183  0.15827236   -0.3022079   0.005579357
## 3     beta_spatial  -0.002155116  0.05679659   -0.1131450  -0.002307947
## 4 beta_interaction  -0.076398486  0.16341841   -0.3997936  -0.076296903
## 5              phi  88.032037971 25.52053275   45.9437600  85.300892040
## 6         deviance -91.741310754  5.96652481 -101.8931332 -92.228905189
##         97.5%     Rhat n.eff
## 1   1.4384549 1.000216 10440
## 2   0.3222417 1.000006 15000
## 3   0.1079505 1.001199  4734
## 4   0.2418551 1.000972  5332
## 5 145.2581238 1.000978  3650
## 6 -78.7751539 1.001411  1788
```

Discussion

Literature Cited